

Recipe·Inspection SW·Modern C++

검사 장비 소프트웨어 구조와 안전한 OpenCV 구현

01 17일차 A. Recipe와 검사 장비 SW 아키텍처

02 17일차 B. Modern C++17과 OpenCV 구현 원칙

원본 20개 Markdown의 내용·코드·표·수식을 보존하여 주제별로 편집

17일차 A. Recipe와 검사 장비 SW 아키텍처

Phase 15 · 권장 학습 시간: 12~16시간 · 난이도: 중급~고급 · 핵심: 재현성, 버전, 추적성

1. 이 Chapter에서 배우는 것

- Recipe가 단순한 설정 파일이 아닌 이유
- Parameter, Calibration, Result, Image의 관계
- 운전 중 설정 변경에도 한 제품의 검사 조건을 고정하는 방법
- 제품 NG와 장비·통신·알고리즘 오류를 구분하는 방법
- 결과를 사후 재현할 수 있는 데이터 구조
- Recipe 검증, 버전 이관, 배포와 되돌리기 전략
- C++17에서 불변 Recipe Snapshot을 사용하는 구조

실무 목표

“어떤 영상이 어떤 조건으로 검사되어 왜 NG가 되었는가?”를 몇 달 후에도 설명할 수 있는 구조를 설계한다. UI의 현재 값이 아니라 **검사 시작 시점에 확정된 Recipe, Calibration, SW Build, 원본 영상**을 결과와 연결하는 것이 핵심이다.

2. 선수 지식

- Inspection Pipeline과 Result
- 좌표계와 Transform Chain
- Image-Machine Calibration
- C++ 구조체, `enum class`, 스마트 포인터의 기본 사용법

3. 핵심 개념

3.1 Recipe란 무엇인가

Recipe는 특정 제품을 동일한 조건으로 검사하기 위한 **버전이 있는 실행 계약**이다.

- 제품과 공정 식별자
- Camera: 노출, Gain, Trigger, Pixel Format
- Illumination: 채널별 밝기와 점등 순서
- Alignment: Mark, 기준 좌표, 탐색 영역, Score 기준
- ROI: 기준 좌표계에서 정의한 검사 영역
- Algorithm: 종류, Parameter, 전처리 순서
- Spec: 상·하한, 판정 규칙, Guard Band
- Calibration 참조 ID와 유효 조건
- 저장 정책: 원본, Overlay, NG 영상, 중간 영상

Recipe는 “현재 화면의 입력값 묶음”과 다르다. 승인·활성화된 Recipe는 직접 수정하지 않고 새 Revision을 만든다.

3.2 구분해야 할 네 가지 버전

구분	의미	변경 예시
Schema Version	파일 구조의 형식	새 필드 추가, 단위 표현 변경
Recipe Revision	제품 검사 조건	Threshold 110 → 118
Calibration ID	좌표·광학 보정 결과	Camera 재장착 후 재교정
Algorithm/SW Build	실행 코드	Blob 필터 버그 수정

Recipe Revision이 같아도 Calibration이나 SW Build가 다르면 결과가 달라질 수 있다. 따라서 Result에는 네 정보를 모두 남겨야 한다.

3.3 기준 좌표로 ROI를 저장한다

ROI를 현재 영상 Pixel에만 저장하면 Glass가 이동하거나 회전했을 때 검사 위치가 틀어진다. ROI는 Recipe Reference 좌표로 저장하고, 매 제품에서 계산한 Align Transform으로 현재 영상에 변환한다.

TEXT

```
Recipe Reference ROI
  | current alignment T
  v
Current Image ROI → inspection
```

좌표의 단위와 기준점을 필드 이름에 드러내야 한다. `x` 보다 `referenceX_mm`, `imageU_px` 가 안전하다.

3.4 불변 Snapshot

검사 도중 작업자가 Threshold를 바꾸면 한 제품의 ROI마다 서로 다른 값이 적용될 수 있다. 제품 검사 시작 시 활성 Recipe의 불변 Snapshot을 얻고, 모든 Stage가 같은 Snapshot을 공유해야 한다.

TEXT

```
Draft 편집 → Validate → Approve → Activate
                        |
Trigger → Snapshot R42 → 모든 검사 Stage → Result(recipe=R42)
                        새 R43 활성화 → 다음 제품부터 적용
```

3.5 NG와 Error는 다른 상태다

- **OK**: 측정이 유효하고 Spec 안이다.
- **NG**: 측정이 유효하지만 Spec 밖이다.
- **Review**: 경계 또는 정책상 작업자 확인이 필요하다.
- **Invalid**: 영상 품질이나 Feature 검출 실패로 측정이 성립하지 않는다.
- **Error**: Camera, 파일, 메모리, 통신, 내부 예외 등 시스템 문제다.

Feature를 찾지 못한 경우가 결함으로 정의된 검사라면 NG일 수 있지만, Align Mark 검출 실패로 검사 위치 자체가 불명확하다면 Invalid 또는 Error 정책이 적절하다. 이 결정은 Recipe에 명시한다.

4. 수식과 정량 설계

4.1 Spec 판정과 Guard Band

측정값 m , 하한 L , 상한 U 일 때 기본 판정은 다음과 같다.

$$OK \Leftrightarrow L \leq m \leq U$$

양쪽 Guard Band g 를 두면 자동 OK 구간은 다음처럼 좁아진다.

$$L + g \leq m \leq U - g$$

고객 Spec이 9.90~10.10 mm이고 $g = 0.02$ mm이면 자동 OK 범위는 9.92~10.08 mm다. 화면 표시값이 아니라 원 측정값으로 판정한 뒤 표시만 반올림한다.

4.2 저장 용량 추정

2,048×2,048 Mono8 원본 한 장은 4.00 MiB다.

$$2048 \times 2048 \times 1 = 4,194,304 \text{ byte} = 4.00 \text{ MiB}$$

제품당 4장, 하루 20,000개를 무압축 저장하면 약 312.5 GiB/day다.

$$4.00 \text{ MiB} \times 4 \times 20,000 = 320,000 \text{ MiB} = 312.5 \text{ GiB}$$

OK는 Sampling, NG는 전량, Error는 원본과 진단 정보를 전량 저장하는 식으로 정책과 보존 기간을 설계한다.

4.3 추적성 최소 Key

TEXT

```
lotId + productId + frameId + cameraId  
+ recipeId/revision + calibrationId + softwareBuild  
+ timestamp + resultId
```

시간만으로 영상을 연결하면 Camera 간 동시 촬영이나 시계 보정 때문에 충돌할 수 있다. 생성 시점에 고유 Result ID를 발급한다.

5. C++17 데이터 모델 예제

CPP · 1/2

```
#include <cstdint>
#include <memory>
#include <optional>
#include <string>
#include <utility>
#include <vector>

enum class Verdict { Ok, Ng, Review, Invalid, Error };
enum class Unit { Pixel, Millimeter, SquareMillimeter, Degree, Dimensionless };

struct RangeSpec {
    double lower{};
    double upper{};
    double guardBand{};
    Unit unit{Unit::Dimensionless};
};

struct CameraRecipe {
    std::string cameraId;
    double exposureUs{};
    double gainDb{};
    int illuminationChannel{};
    double illuminationPercent{};
};

struct RoiRecipe {
    std::string roiId;
    std::vector<std::pair<double, double>> referencePolygonMm;
    std::string inspectorType;
    RangeSpec spec;
};

struct InspectionRecipe {
    std::uint32_t schemaVersion{};
    std::string recipeId;
    std::uint32_t revision{};
    std::string productCode;
    std::string calibrationId;
    std::vector<CameraRecipe> cameras;
    std::vector<RoiRecipe> rois;
};
```

CPP · 2/2

```
struct Measurement {
    std::string name;
    std::optional<double> value;
    Unit unit{Unit::Dimensionless};
    RangeSpec appliedSpec;
    Verdict verdict{Verdict::Invalid};
    std::string reasonCode;
};

struct InspectionResult {
    std::string resultId;
    std::string lotId;
    std::string productId;
    std::string recipeId;
    std::uint32_t recipeRevision{};
    std::string calibrationId;
    std::string softwareBuild;
    Verdict overall{Verdict::Invalid};
    std::vector<Measurement> measurements;
    std::vector<std::string> imagePaths;
};
```

`Measurement::value` 가 없을 수 있으므로 0.0 같은 가짜 값을 넣지 않는다. 값 부재와 실제 0은 의미가 다르다.

5.1 Recipe 의미 검증

C++

```
#include <cmath>
#include <string>
#include <vector>

struct ValidationIssue { std::string path; std::string message; };

std::vector<ValidationIssue> validateRecipe(const InspectionRecipe& recipe) {
    std::vector<ValidationIssue> issues;
    if (recipe.recipeId.empty()) issues.push_back({"recipeId", "must not be empty"});
    if (recipe.calibrationId.empty()) issues.push_back({"calibrationId", "is required"});

    for (std::size_t i = 0; i < recipe.cameras.size(); ++i) {
        const auto& camera = recipe.cameras[i];
        const std::string base = "cameras[" + std::to_string(i) + "]";
        if (!std::isfinite(camera.exposureUs) || camera.exposureUs <= 0.0)
            issues.push_back({base + ".exposureUs", "must be finite and positive"});
        if (!std::isfinite(camera.gainDb))
            issues.push_back({base + ".gainDb", "must be finite"});
        if (!std::isfinite(camera.illuminationPercent) ||
            camera.illuminationPercent < 0.0 || camera.illuminationPercent > 100.0)
            issues.push_back({base + ".illuminationPercent", "must be in [0, 100]"});
    }
    for (std::size_t i = 0; i < recipe.rois.size(); ++i) {
        const auto& roi = recipe.rois[i];
        const std::string base = "rois[" + std::to_string(i) + "]";
        if (roi.referencePolygonMm.size() < 3U)
            issues.push_back({base + ".referencePolygonMm", "needs at least 3 points"});
        if (!(roi.spec.lower <= roi.spec.upper))
            issues.push_back({base + ".spec", "lower must not exceed upper"});
        const double halfWidth = (roi.spec.upper - roi.spec.lower) * 0.5;
        if (!std::isfinite(roi.spec.guardBand) ||
            roi.spec.guardBand < 0.0 || roi.spec.guardBand > halfWidth)
            issues.push_back({base + ".spec.guardBand", "is outside valid range"});
    }
    return issues;
}
```

Parser가 성공했다는 사실은 Recipe가 유효하다는 뜻이 아니다. NaN, 무한대, 범위 역전, 빈 ROI, 중복 ID, 지원하지 않는 Camera 조합까지 검증한다.

5.2 불변 Snapshot 저장소

CPP

```
#include <memory>
#include <mutex>
#include <stdexcept>
#include <utility>

class RecipeStore {
public:
    void activate(InspectionRecipe recipe) {
        const auto issues = validateRecipe(recipe);
        if (!issues.empty())
            throw std::invalid_argument("invalid recipe: " + issues.front().path);

        auto next = std::make_shared<const InspectionRecipe>(std::move(recipe));
        std::lock_guard<std::mutex> lock(mutex_);
        active_ = std::move(next);
    }

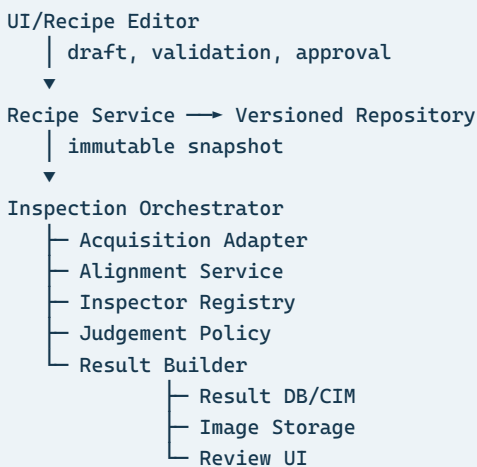
    std::shared_ptr<const InspectionRecipe> snapshot() const {
        std::lock_guard<std::mutex> lock(mutex_);
        if (!active_) throw std::runtime_error("no active recipe");
        return active_;
    }

private:
    mutable std::mutex mutex_;
    std::shared_ptr<const InspectionRecipe> active_;
};
```

검사 Thread는 시작할 때 Snapshot을 한 번 받아 완료할 때까지 유지한다. 실제 장비에서는 Camera/조명 변경과 Recipe 활성화를 제품 사이 Safe Point에서 원자적으로 묶어야 한다.

6. SW 아키텍처

TEXT



- UI는 검사 알고리즘을 직접 호출하지 않는다.
- Inspector는 화면이나 DB에 의존하지 않고 입력, Parameter, Calibration을 받아 결과를 반환한다.
- Result Builder가 공통 추적 정보를 한 번만 확정한다.
- 저장 실패가 검사 판정을 바꾸지 않도록 판정과 전송 상태를 분리한다.

6.1 Transactional Activation

1. Draft를 별도 공간에서 편집한다.
2. Schema와 의미 검증을 수행한다.
3. Camera/조명/Calibration의 존재와 호환성을 확인한다.
4. 장비가 제품 사이 Safe Point인지 확인한다.
5. 하드웨어 설정 적용에 성공하면 Snapshot을 활성화한다.
6. 일부 적용 실패 시 이전 설정으로 복구하고 Error를 기록한다.

6.2 Result는 append-only로 취급한다

재판정이 필요하면 기존 Result를 덮어쓰지 말고 `parentResultId`, 재판정 이유, 새 Recipe/SW 버전을 가진 새 Result를 생성한다.

7. Recipe 파일과 이관 전략

- 단위를 명시한다. `exposureUs`, `diameterMm` 처럼 이름에 포함한다.
- 알 수 없는 필드와 누락 필드 정책을 정한다.
- 저장 전 정규화하고 Hash 또는 서명을 남긴다.
- Schema Migration은 `v1 → v2 → v3` 처럼 단계별 순수 변환으로 만든다.
- Migration 후 반드시 현재 Validator를 다시 실행한다.
- 원본 파일과 변환 결과를 모두 추적 가능하게 보관한다.

TEXT

```
load bytes → parse → schema migrate → semantic validate
           → dependency validate → immutable domain object
```

파일명만 같고 내용이 바뀌는 문제를 막으려면 활성화 시 계산한 Content Hash도 Result에 넣는 것이 좋다.

8. 결과와 영상 저장 정책

- Raw: 재검증이 가능한 원본. 가능하면 압축 전 Bit Depth 보존
- Display: Review UI용 밝기 변환 영상
- Overlay: ROI, Feature, 수치, 판정을 그린 파생 영상
- Intermediate: 문제 분석을 위한 Threshold/Edge/Mask 등

Overlay만 저장하면 원 알고리즘을 재실행할 수 없다. 저장 실패가 발생해도 검사 결과 NG를 OK로 바꾸지 않고 보관 상태를 `StorageError` 로 별도 기록한다. 장비 정지·알람 여부는 운영 정책으로 결정한다.

9. 실무 실패 사례와 진단

사례 1. 같은 제품인데 재검사 결과가 다르다

- 확인: Raw, Recipe Hash, Calibration ID, SW Build, OpenCV/SDK 버전
- 원인: UI의 현재 Recipe로 재실행, 자동 노출, 비결정적 병렬 순서
- 대책: Result가 참조한 Snapshot으로 Offline Replay한다.

사례 2. Recipe 변경 직후 몇 개 제품만 이상하다

- 확인: Trigger/활성화 시각, Frame별 Revision, Camera 설정 ACK
- 원인: 운전 중 객체 직접 수정, Camera와 Algorithm 적용 시점 불일치
- 대책: 불변 Snapshot과 Safe Point Activation을 사용한다.

사례 3. Calibration이 바뀌었는데 이전 ROI를 쓴다

- 확인: Recipe 요구 Calibration ID와 실제 활성화 ID
- 원인: ID가 아닌 최신 파일명만 참조
- 대책: 명시적 ID와 Camera/렌즈/해상도 호환성을 확인한다.

사례 4. 0.0 값이 정상 측정인지 실패인지 모른다

- 원인: 실패를 숫자 Sentinel로 표현
- 대책: `optional`, Verdict, Reason Code를 별도 필드로 둔다.

10. 실습 과제

1. Camera 1대, 조명 2채널, Align Mark 3개, Dynamic ROI 4개인 Glass 검사 Recipe를 설계한다.
2. Scratch 초과, Mark 미검출, Camera Timeout, Calibration 만료, DB 단절을 NG/Invalid/Error로 분류하고 장비 동작을 정한다.
3. 과거 한 제품을 현재 UI 값 없이 재실행하기 위한 입력과 버전을 나열한다.
4. Recipe 활성화 일부 실패 시 Rollback Sequence를 상태로 그린다.

11. 이해 점검 문제

1. Schema Version과 Recipe Revision의 차이는 무엇인가?
2. 검사 시작 시 Snapshot을 고정해야 하는 이유는 무엇인가?
3. ROI를 Reference 좌표로 저장하는 이유는 무엇인가?
4. Feature 미검출이 항상 NG가 아닌 이유는 무엇인가?
5. Result에 Calibration ID와 SW Build가 필요한 이유는 무엇인가?
6. Overlay만 저장했을 때 재현성 문제는 무엇인가?
7. Recipe 활성화를 Safe Point에서 수행해야 하는 이유는 무엇인가?
8. 저장 실패와 검사 판정을 분리해야 하는 이유는 무엇인가?

12. 면접 질문 6개

Q1. Recipe를 어떻게 정의하겠습니까?

- 초보자: 제품별 검사 설정의 묶음입니다.
- 실무자: Camera·조명·Align·ROI·Algorithm·Spec·Calibration 참조를 포함한 버전 있는 실행 계약입니다.
- 30초 답변: 승인 Recipe를 불변 Snapshot으로 적용하고 Revision·Calibration·SW Build를 Result에 남겨 재현성을 확보합니다.

Q2. 운전 중 Recipe 변경을 어떻게 처리합니까?

- 초보자: 검사가 끝난 다음 변경합니다.
- 실무자: Draft 검증 후 Safe Point에서 하드웨어 설정과 Snapshot을 원자적으로 전환합니다.
- 30초 답변: 현재 제품은 기존 Snapshot으로 끝내고 다음 Trigger부터 새 Revision을 사용합니다.

Q3. NG와 Error의 차이는 무엇입니까?

- 초보자: NG는 제품 불량이고 Error는 장비 문제입니다.
- 실무자: NG는 유효 측정의 Spec 위반이고 Error는 검사가 정상 수행되지 못한 상태입니다.
- 30초 답변: Invalid/Error는 별도 Reason Code와 장비 정책을 가져야 합니다.

Q4. 결과 재현에 어떤 정보가 필요합니까?

- 초보자: 원본 영상과 검사 조건입니다.
- 실무자: Raw, Recipe, Calibration, SW Build, Camera 조건과 입력 ID가 필요합니다.
- 30초 답변: Result ID를 기준으로 원본과 네 가지 버전을 고정해 Offline Replay가 가능해야 합니다.

Q5. Recipe 파일을 읽었으면 바로 적용해도 됩니까?

- 초보자: 형식이 맞는지 확인해야 합니다.
- 실무자: Parse 외에 수치 범위, ID, ROI, Calibration/Camera 호환성까지 검증합니다.
- 30초 답변: Parse→Migration→의미/의존성 검증 후 불변 객체로 활성화합니다.

Q6. 검사 영상 저장 정책은 어떻게 정합니까?

- 초보자: NG 영상을 우선 저장합니다.
- 실무자: 데이터량, 추적성, 보존 기간을 고려해 OK Sampling과 NG/Error 저장을 구분합니다.
- 30초 답변: Raw는 재실행용, Overlay는 Review용으로 분리하고 일일 용량과 장애 정책까지 설계합니다.

다음 학습

17일차 B. Modern C++17과 OpenCV 구현 원칙에서 메모리 수명, RAII, 오류 처리, 동시성, 테스트 관점으로 구현한다.

17일차 B. Modern C++17과 OpenCV 구현 원칙

Phase 16 · 권장 학습 시간: 12~16시간 · 난이도: 중급~고급 · 핵심: 수명, 소유권, 실패 처리, 테스트

1. 이 Chapter에서 배우는 것

- 장비 SW에서 RAII가 중요한 이유
- `cv::Mat` 의 얇은 복사와 Camera Buffer 수명
- `const` , `enum class` , 스마트 포인터 선택 기준
- 제품 NG와 예외를 분리하는 오류 처리
- Thread 간 Frame 전달과 Backpressure
- 불필요한 복사 없이 안전하게 성능을 확보하는 방법
- 영상 알고리즘의 Unit/Integration/Replay Test 설계

실무 목표

빠른 Demo가 아니라 장시간 운전해도 누수, Use-after-free, 조건 혼합, 무한 Queue가 발생하지 않는 검사 엔진의 구현 원칙을 익힌다.

2. 선수 지식

- Recipe와 SW Architecture
- Inspection Pipeline
- C++17 기본 문법과 OpenCV `cv::Mat`

3. 소유권과 RAII

Pointer를 고르기 전에 생성자, 파괴자, 공유 범위, 비동기 수명을 결정한다.

- 값 또는 `unique_ptr` : 소유자가 하나일 때 기본 선택
- `shared_ptr` : 실제로 수명을 공유해야 할 때만 사용
- Reference/Raw Pointer: 소유하지 않는 짧은 접근
- `weak_ptr` : 공유 객체를 관찰하지만 수명을 늘리지 않을 때

`shared_ptr` 를 무조건 사용하면 순환 참조와 불명확한 파괴 시점이 생긴다.

3.1 Camera Resource를 RAII로 관리한다

C++

```
#include <stdexcept>

class ICameraSdk {
public:
    using Handle = void*;
    virtual ~ICameraSdk() = default;
    virtual Handle open(int index) = 0;
    virtual void start(Handle handle) = 0;
    virtual void stop(Handle handle) noexcept = 0;
    virtual void close(Handle handle) noexcept = 0;
};

class CameraSession {
public:
    CameraSession(ICameraSdk& sdk, int index) : sdk_(sdk), handle_(sdk.open(index)) {
        if (handle_ == nullptr) throw std::runtime_error("camera open failed");
        try {
            sdk_.start(handle_);
            started_ = true;
        } catch (...) {
            sdk_.close(handle_);
            handle_ = nullptr;
            throw;
        }
    }

    ~CameraSession() noexcept {
        if (handle_ == nullptr) return;
        if (started_) sdk_.stop(handle_);
        sdk_.close(handle_);
    }

    CameraSession(const CameraSession&) = delete;
    CameraSession& operator=(const CameraSession&) = delete;
    ICameraSdk::Handle handle() const noexcept { return handle_; }

private:
    ICameraSdk& sdk_;
    ICameraSdk::Handle handle_{nullptr};
    bool started_{false};
};
```

Destructor는 예외를 던지지 않는다. SDK가 실패 코드를 반환한다면 Destructor에서는 Log만 남기고, 명시적 `shutdown()` 에서 오류를 처리하는 방식을 고려한다.

4. cv::Mat 과 Buffer 수명

4.1 얇은 복사와 깊은 복사

CPP

```
#include <opencv2/imgcodecs.hpp>

cv::Mat a = cv::imread("input.png", cv::IMREAD_GRAYSCALE);
cv::Mat b = a;           // pixel buffer 공유
cv::Mat c = a.clone();   // 독립 pixel buffer

b.at<unsigned char>(0, 0) = 0; // a의 같은 pixel도 변경됨

cv::Mat roiView = a(cv::Rect(10, 10, 100, 80)); // a와 buffer 공유
cv::Mat roiCopy = roiView.clone();             // 독립 buffer
```

OpenCV가 소유하는 `Mat` 은 Reference Count로 Buffer 수명을 유지한다. Camera SDK가 소유한 외부 메모리를 감싼 `Mat` 은 SDK가 Buffer를 재사용하면 픽셀이 바뀐다.

4.2 외부 Camera Buffer를 감쌀 때

CPP

```
#include <cstddef>
#include <opencv2/core.hpp>
#include <stdexcept>

cv::Mat wrapMono8(unsigned char* data, int width, int height, std::size_t stride) {
    if (data == nullptr || width <= 0 || height <= 0 ||
        stride < static_cast<std::size_t>(width)) {
        throw std::invalid_argument("invalid camera buffer");
    }
    return cv::Mat(height, width, CV_8UC1, data, stride);
}

cv::Mat acquireOwnedCopy(unsigned char* data, int width, int height,
                          std::size_t stride) {
    return wrapMono8(data, width, height, stride).clone();
}
```

비동기 Pipeline에는 다음 중 하나를 사용한다.

- 즉시 `clone()` 하여 애플리케이션이 소유
- SDK Buffer Handle과 수명 객체를 함께 전달하고 완료 후 반환
- 고정 크기 Buffer Pool로 복사 비용과 수명 관리

SDK Callback이 끝난 뒤 무효가 되는 Pointer만 Queue에 넣으면 안 된다.

4.3 const 의 한계

입력을 수정하지 않는 함수는 `const cv::Mat&` 로 받는다. 다만 Mat Header의 `const` 가 공유 Pixel의 완전한 불변성을 보장하지는 않으므로, 입력을 수정하지 않고 출력 Mat을 분리한다는 팀 규칙이 필요하다.

CPP

```
#include <opencv2/core.hpp>
#include <vector>

struct BlobFeature { double areaPx{}; cv::Point2d centerPx{}; };
std::vector<BlobFeature> findBlobs(const cv::Mat& binary);
```

5. Frame Packet과 시간

CPP

```
#include <chrono>
#include <cstdint>
#include <memory>
#include <opencv2/core.hpp>
#include <stdexcept>
#include <string>
#include <utility>

struct InspectionRecipe;

struct FramePacket {
    std::uint64_t frameId{};
    std::string cameraId;
    std::chrono::steady_clock::time_point acquiredAt;
    cv::Mat image;
    std::shared_ptr<const InspectionRecipe> recipe;
};

FramePacket makePacket(std::uint64_t id, std::string cameraId,
                      const cv::Mat& ownedImage,
                      std::shared_ptr<const InspectionRecipe> recipe) {
    if (ownedImage.empty()) throw std::invalid_argument("empty frame");
    if (!recipe) throw std::invalid_argument("null recipe snapshot");
    return {id, std::move(cameraId), std::chrono::steady_clock::now(),
            ownedImage, std::move(recipe)};
}
```

ownedImage 는 OpenCV가 소유한 Buffer라는 전제다. 외부 Buffer라면 복제하거나 SDK 수명 객체를 Packet에 함께 보관한다.

- Cycle Time: 역행하지 않는 **steady_clock**
- 외부 추적 Timestamp: UTC 기반 **system_clock**

NTP 보정으로 System Clock이 움직일 수 있으므로 처리 시간 측정에는 쓰지 않는다.

6. 오류 처리

6.1 예외와 Domain Result를 구분한다

- 예외/Error: Camera Open, 파일 I/O, 잘못된 Recipe, 불변 조건 위반
- 정상 Domain 결과: Spec 초과, Scratch 존재, Review 판정
- 예상 가능한 검출 실패: Status/Reason Code로 표현하고 정책에서 NG/Invalid 결정

CPP

```
#include <optional>
#include <string>

enum class DetectionStatus { Found, NotFound, Ambiguous, InvalidInput };

struct CircleDetection {
    DetectionStatus status{DetectionStatus::InvalidInput};
    std::optional<cv::Point2d> centerPx;
    std::optional<double> radiusPx;
    double score{};
    std::string reason;
};
```

`catch (...)` 뒤 OK나 0.0을 반환하면 결함이 은폐된다. Component 경계에서 Stage, Frame ID, 원인 정보를 추가하고 Result Error로 변환한다.

6.2 assert 와 실행 검증

`assert` 는 Release Build에서 사라질 수 있다. 외부 입력, Recipe, Camera Frame은 항상 실행 중 검증하고, `assert` 는 개발자가 보장해야 하는 내부 불변 조건에만 사용한다.

7. 동시성과 Backpressure

입력 12 fps, 처리 10 fps, Frame 4 MiB라면 10분 후 Queue에 약 4.69 GiB가 쌓인다.

$$(12 - 10) \text{ frame/s} \times 600 \text{ s} \times 4 \text{ MiB} = 4800 \text{ MiB} \approx 4.69 \text{ GiB}$$

Queue는 상한과 Full 정책을 가진다.

- Trigger를 늦추거나 장비를 Interlock으로 정지
- Preview Frame만 Drop
- 검사 Frame은 조용히 버리지 않고 Error 처리

7.1 Thread 경계 원칙

- 소유권이 명확한 `FramePacket` 을 전달한다.
- Recipe는 `shared_ptr<const ...>` Snapshot으로 공유한다.
- UI 객체를 Worker Thread에서 직접 갱신하지 않는다.
- Queue Close 상태와 `notify_all` 로 종료 대기를 해제한다.
- 순서가 중요하면 `productId/frameId` 로 결과를 재정렬한다.

제품 Thread Pool과 OpenCV 내부 Thread를 동시에 최대로 쓰면 Oversubscription이 생길 수 있다. 실제 CPU에서 조정하고 평균뿐 아니라 P95/P99 Cycle Time을 본다.

8. 성능 최적화와 계산

1. 요구 Cycle Time, 영상 크기, Camera 수를 수치로 정한다.
2. Stage별 시간을 측정한다.
3. 가장 큰 병목을 확인한다.
4. ROI 축소, Color 변환 제거, Buffer 재사용을 적용한다.
5. 정확도 회귀 테스트 후 P95/P99와 메모리를 다시 측정한다.

4,096×3,000 Mono8 한 장은 11.71875 MiB다. 20 fps에서 한 번 불필요하게 전체 복사하면 최소 234.375 MiB/s가 추가된다.

$$4096 \times 3000 / 2^{20} = 11.71875 \text{ MiB}$$

$$11.71875 \times 20 = 234.375 \text{ MiB/s}$$

그러나 안전하지 않은 Zero-copy보다 올바른 한 번의 복사가 낫다. 먼저 수명을 보장한 뒤 Profile로 최적화한다.

9. 테스트 전략

9.1 Unit Test

- Spec: `L` , `U` , `L±ε` , `U±ε`
- 좌표 변환: 알려진 Translation/Rotation과 역변환
- Blob: 합성 원·사각형의 면적과 중심
- Recipe Validator: 누락, NaN, 범위 역전, 중복 ID
- 판정 정책: Found/NotFound/Ambiguous/Error 조합

9.2 Golden Image와 Replay

고정 영상과 Recipe로 Feature, Measurement, Verdict를 비교한다. Library 차이를 고려해 중심 오차 ≤ 0.1 px, 면적 상대 오차 $\leq 0.5\%$ 처럼 의미 있는 허용오차를 정의한다. 실제 생산 Raw와 당시 Snapshot으로 Offline Pipeline을 재실행하는 Replay Test도 둔다.

9.3 Integration Test

- Camera Timeout과 재연결
- Trigger 누락·중복·순서 변경
- Disk Full과 DB 단절
- Recipe 전환 중 Trigger
- 종료 중 Queue에 남은 Frame
- Calibration/해상도 불일치

GUI Click, 성공 영상, 화면 반올림 값, 고정 `sleep`에만 의존하는 Test는 피한다.

10. 테스트 가능한 Inspector

C++

```
#include <opencv2/core.hpp>
#include <stdexcept>

struct ThresholdParameters { double threshold{}; bool invert{}; };

class IThresholdOperator {
public:
    virtual ~IThresholdOperator() = default;
    virtual cv::Mat apply(const cv::Mat& gray,
                        const ThresholdParameters& parameters) const = 0;
};

class AreaInspector {
public:
    explicit AreaInspector(const IThresholdOperator& op) : op_(op) {}

    double measureForegroundAreaPx(const cv::Mat& gray,
                                const ThresholdParameters& parameters) const {
        if (gray.empty() || gray.type() != CV_8UC1)
            throw std::invalid_argument("expected non-empty CV_8UC1 image");
        const cv::Mat binary = op_.apply(gray, parameters);
        if (binary.empty() || binary.type() != CV_8UC1 || binary.size() != gray.size())
            throw std::runtime_error("threshold operator returned invalid image");
        return static_cast<double>(cv::countNonZero(binary));
    }

private:
    const IThresholdOperator& op_;
};
```

작은 Interface를 주입하면 Test Double로 Inspector의 입력 검증과 계산을 분리해 검사할 수 있다. 실제 영상 알고리즘은 Golden Image Test로 추가 검증한다.

11. 실패 사례와 실습

자주 발생하는 실패

- 영상이 가끔 섞임: Callback Buffer를 Header만 감싸 Queue에 전달
- 장시간 후 메모리 증가: 무한 Queue, `shared_ptr` 순환, Cache 무제한
- 종료 시 멈춤: Queue Close/ `notify_all` /Thread Join 순서 오류
- Debug/Release 차이: 미초기화 값, Data Race, Release에서 사라지는 `assert`

실습 과제

1. 사용 중인 Camera SDK의 Callback Buffer 수명 규칙을 확인한다.
2. Pipeline의 모든 `cv::Mat` 에 Owner와 Lifetime을 표로 작성한다.
3. 입력 15 fps, 처리 12 fps, Frame 8 MiB인 Queue의 5분 증가량을 계산한다. 정답은 7,200 MiB, 약 7.03 GiB다.
4. Recipe 전환과 Trigger가 동시에 발생하는 Integration Test를 설계한다.
5. 같은 Raw/Recipe를 100회 Replay해 결과 변동과 P99 시간을 측정한다.

이해 점검

1. 얇은 복사와 `clone()` 의 차이는 무엇인가?
2. 외부 Buffer Mat이 위험한 이유는 무엇인가?
3. `shared_ptr` 가 항상 기본값이 아닌 이유는 무엇인가?
4. Destructor에서 예외를 던지지 않는 이유는 무엇인가?
5. 제품 NG를 Exception으로 표현하지 않는 이유는 무엇인가?
6. 검사 Queue에 상한이 필요한 이유는 무엇인가?

12. 면접 질문 6개

Q1. RAII가 장비 SW에서 왜 중요하니까?

- 초보자: 객체가 사라질 때 Resource를 자동 해제합니다.
- 실무자: Camera, 파일, Lock을 모든 Return과 예외 경로에서 결정적으로 정리합니다.
- 30초 답변: Resource 수명을 객체와 묶고 복사를 금지해 소유자를 명확히 하며 Destructor는 안전하게 정리합니다.

Q2. `cv::Mat` 을 다른 Thread로 넘길 때 무엇을 봅니까?

- 초보자: 영상이 비어 있지 않은지 봅니다.
- 실무자: Buffer Owner, Reference Count, SDK 재사용 시점, Writer 존재를 확인합니다.
- 30초 답변: OpenCV 소유 Buffer는 수명을 공유할 수 있지만 외부 Buffer는 Clone이나 Handle 수명 전달이 필요합니다.

Q3. 검사 실패를 모두 Exception으로 처리하니까?

- 초보자: 제품 NG와 프로그램 오류를 나눕니다.
- 실무자: Detection/판정은 Domain Result로, I/O와 불변 조건 위반은 Error로 처리합니다.
- 30초 답변: 정상 NG 흐름과 시스템 실패를 분리해야 통계와 알람이 정확합니다.

Q4. Queue 증가를 어떻게 막습니까?

- 초보자: Queue 크기를 제한합니다.
- 실무자: Bounded Queue, Backpressure, Trigger 제어, 종류별 Drop 정책을 둡니다.
- 30초 답변: 생산 Frame을 조용히 버리지 않고 Queue Full을 Error/Interlock으로 연결합니다.

Q5. 영상 알고리즘은 어떻게 테스트하니까?

- 초보자: 여러 영상을 넣어 확인합니다.
- 실무자: 합성 Unit, Golden Image, 실제 Replay, 장애 Integration Test를 구성합니다.
- 30초 답변: UI와 분리하고 고정 Raw-Recipe로 수치와 Verdict를 자동 비교합니다.

Q6. 성능 최적화는 어디서 시작합니까?

- 초보자: 느린 부분을 측정합니다.
- 실무자: Stage별 평균/P95/P99와 메모리-Queue를 측정합니다.
- 30초 답변: 정확도 회귀 Test를 고정한 뒤 가장 큰 병목부터 개선하고 다시 측정합니다.

다음 학습

다음 문서에서는 1~17일차의 Camera·광학·영상처리·Align-Calibration·검사 구조를 면접 질문과 장애 분석 시나리오로 통합한다.